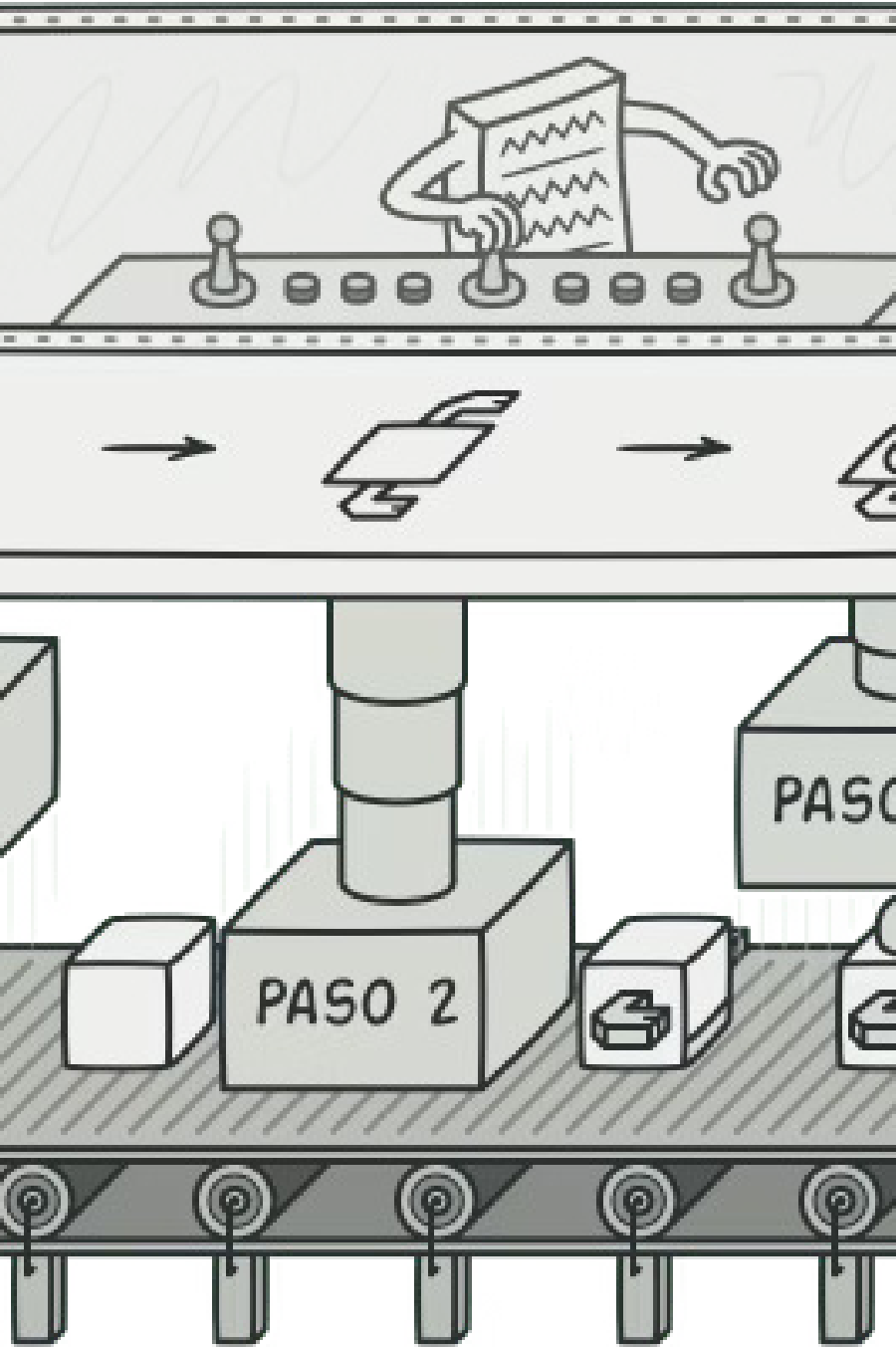




PATRÓN DE DISEÑO CREAMACIONAL

Builder (Constructor)

Un patrón de diseño que nos permite construir objetos complejos paso a paso, produciendo distintos tipos y representaciones empleando el mismo código de construcción.

Agenda de la Presentación

01

Propósito del Patrón

Qué es Builder y para qué sirve

03

La Solución

Separar la construcción en pasos independientes

05

Aplicabilidad e Implementación

Cuándo usarlo y cómo implementarlo

02

El Problema

Constructores telescópicos y explosión de subclases

04

Estructura y Pseudocódigo

Diagrama UML y ejemplo con automóviles

06

Pros, Contras y Relaciones

Ventajas, desventajas y vínculos con otros patrones

Propósito del Patrón Builder

Builder es un **patrón de diseño creacional** que nos permite construir objetos complejos paso a paso. El patrón nos permite producir **distintos tipos y representaciones** de un objeto empleando el mismo código de construcción.

En lugar de crear un objeto en una sola llamada, Builder descompone el proceso en pasos discretos y controlables, otorgando flexibilidad total sobre el resultado final.

El Problema

Imagina un objeto complejo que requiere una inicialización laboriosa, paso a paso, de muchos campos y objetos anidados. Normalmente, este código de inicialización está sepultado dentro de un **monstruoso constructor** con una gran cantidad de parámetros. O, peor aún: disperso por todo el código cliente.

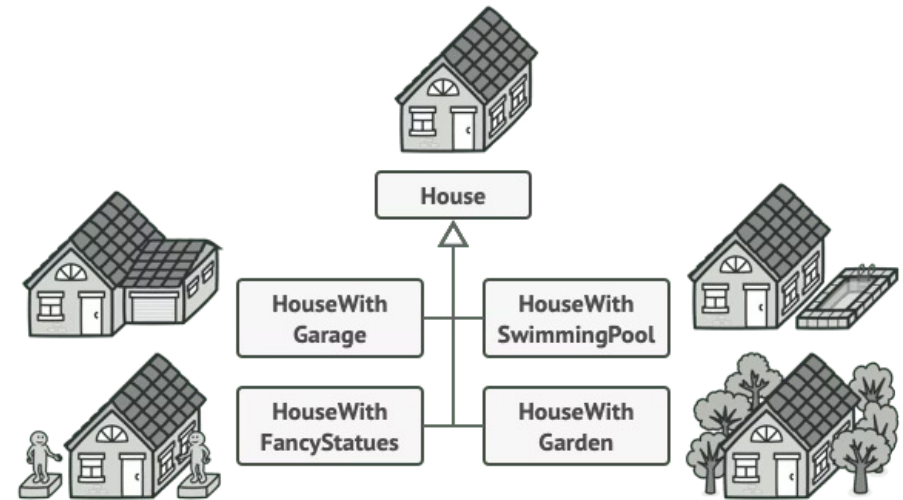
Ejemplo: Construir una Casa

Para construir una casa sencilla, debemos construir cuatro paredes y un piso, instalar una puerta, colocar ventanas y ponerle un tejado.

Pero ¿qué pasa si quieres una casa más grande y luminosa, con un jardín y otros extras como:

- Sistema de calefacción
- Instalación de fontanería
- Cableado eléctrico

La complejidad crece rápidamente con cada nueva característica.



Solución Ingenua #1: Explosión de Subclases

La solución más sencilla es extender la clase base `Casa` y crear un grupo de subclases que cubran todas las combinaciones posibles de los parámetros.

Problema

Acabarás con una cantidad considerable de subclases

Escalabilidad

Cualquier parámetro nuevo (ej. estilo del porche) exigirá incrementar la jerarquía aún más

Mantenimiento

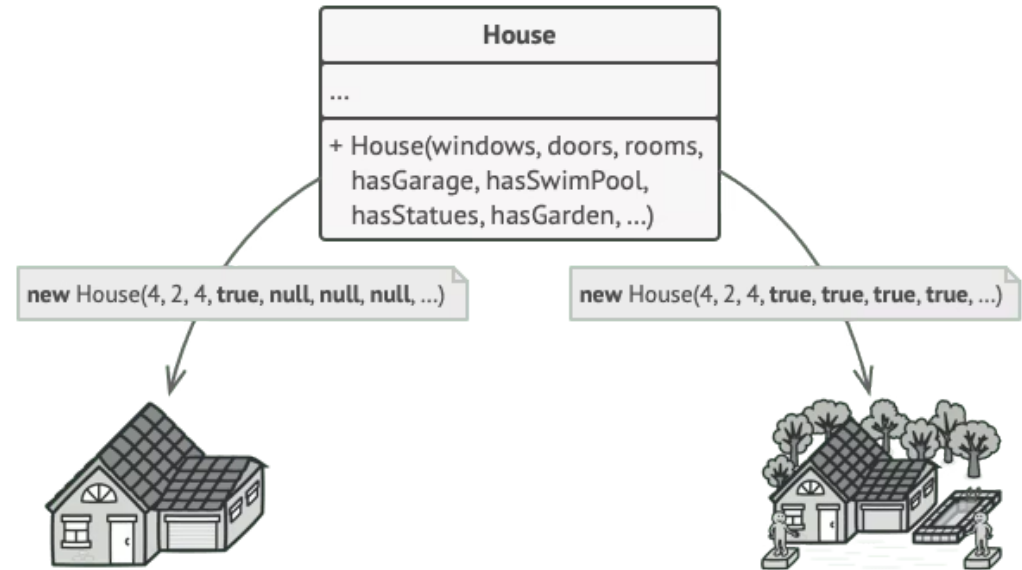
Crear una subclase por cada configuración posible complica demasiado el programa

Solución Ingenua #2: Constructor Telescópico

Otra posibilidad: crear un **enorme constructor** dentro de la clase base Casa con todos los parámetros posibles para controlar el objeto.

Aunque elimina la necesidad de subclases, genera otro problema: **no todos los parámetros son necesarios todo el tiempo**.

Por ejemplo, solo una pequeña parte de las casas tiene piscina, por lo que los parámetros relacionados con piscinas serán inútiles en nueve de cada diez casos.

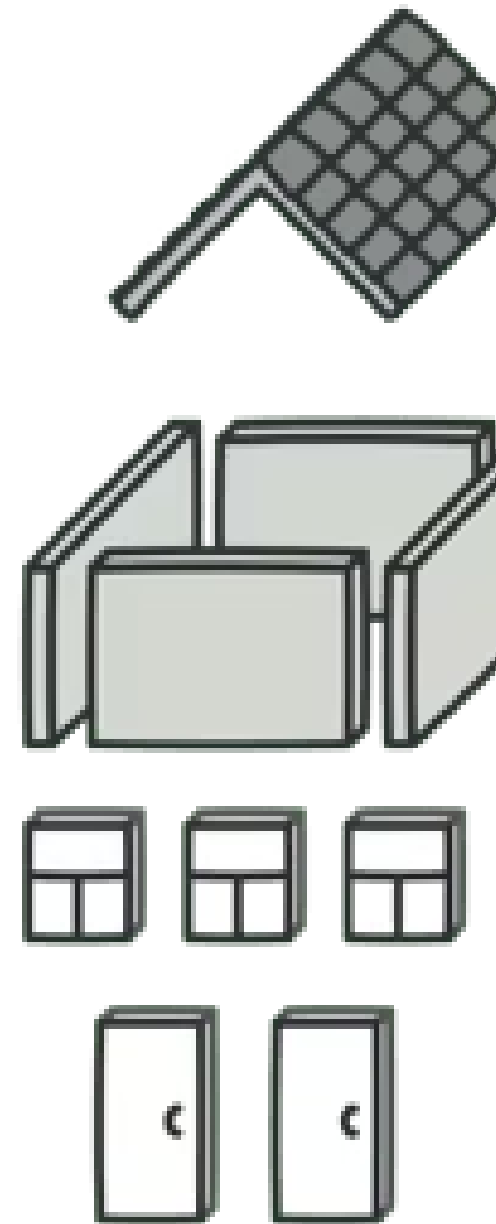
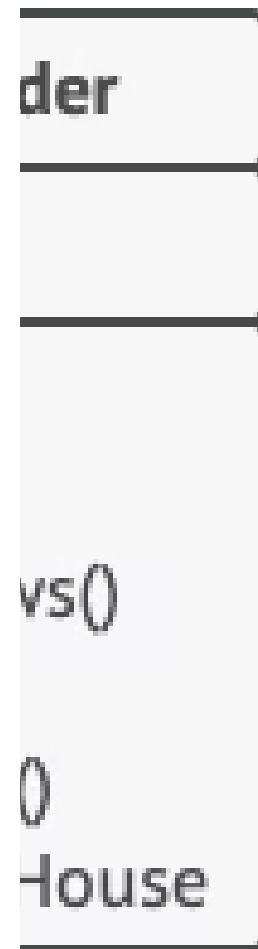


Un constructor con un montón de parámetros tiene su inconveniente: las llamadas al constructor son bastante feas.

La Solución: Patrón Builder

El patrón Builder sugiere que **saques el código de construcción del objeto de su propia clase** y lo coloques dentro de objetos independientes llamados **constructores**.

El patrón Builder te permite construir objetos complejos paso a paso. No permite a otros objetos acceder al producto mientras se construye.



Construcción Paso a Paso

El patrón organiza la construcción de objetos en una serie de pasos (`construirParedes`, `construirPuerta`, etc.). Para crear un objeto, se ejecuta una serie de estos pasos en un objeto constructor.

- ❏ Lo importante es que **no necesitas invocar todos los pasos**. Puedes invocar sólo aquellos que sean necesarios para producir una configuración particular de un objeto.

Algunos pasos de la construcción pueden necesitar una implementación diferente cuando tengamos que construir distintas representaciones del producto. Por ejemplo, las paredes de una cabaña pueden ser de madera, pero las de un castillo tienen que ser de piedra.

Múltiples Constructores, Mismos Pasos

Podemos crear varias clases constructoras distintas que implementen la misma serie de pasos de construcción, pero de forma diferente. Al invocar la misma serie de pasos, obtenemos productos distintos.

1

Constructor 1

Madera y vidrio → Casa normal

2

Constructor 2

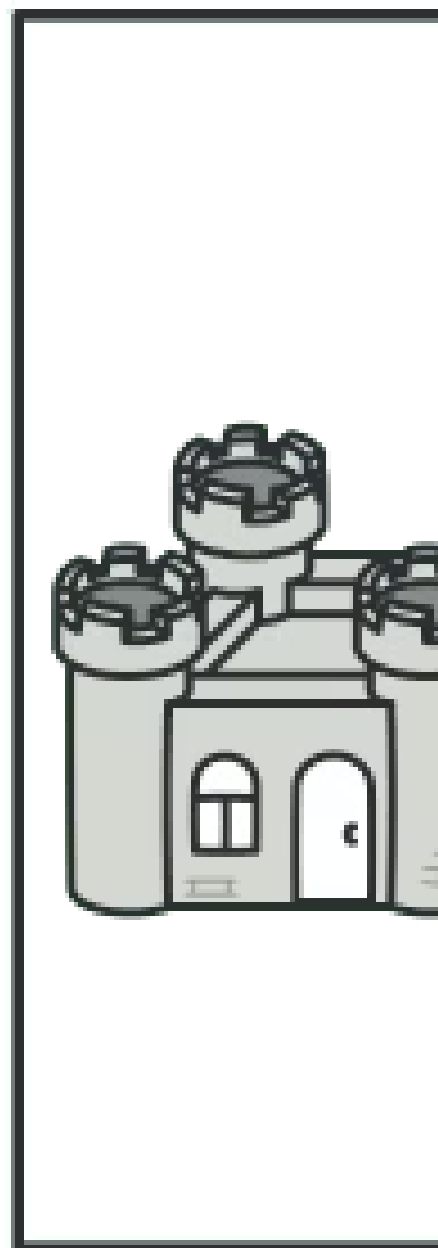
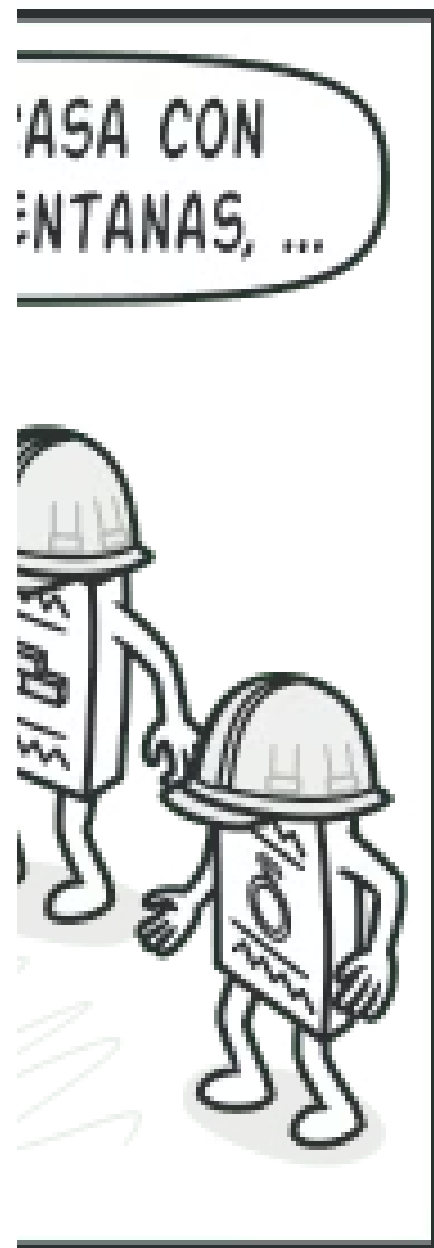
Piedra y hierro → Pequeño castillo

3

Constructor 3

Oro y diamantes → Palacio

Esto sólo funciona si el código cliente que invoca los pasos de construcción es capaz de interactuar con los constructores mediante una **interfaz común**.



La Clase Directora

Puedes extraer una serie de llamadas a los pasos del constructor y ponerlas en una clase independiente llamada **directora**.

La clase directora define el **orden** en el que se deben ejecutar los pasos de construcción, mientras que el constructor proporciona la **implementación** de dichos pasos.

¿Es obligatoria?

No es estrictamente necesario tener una clase directora, pero es un buen lugar donde colocar distintas rutinas de construcción reutilizables.

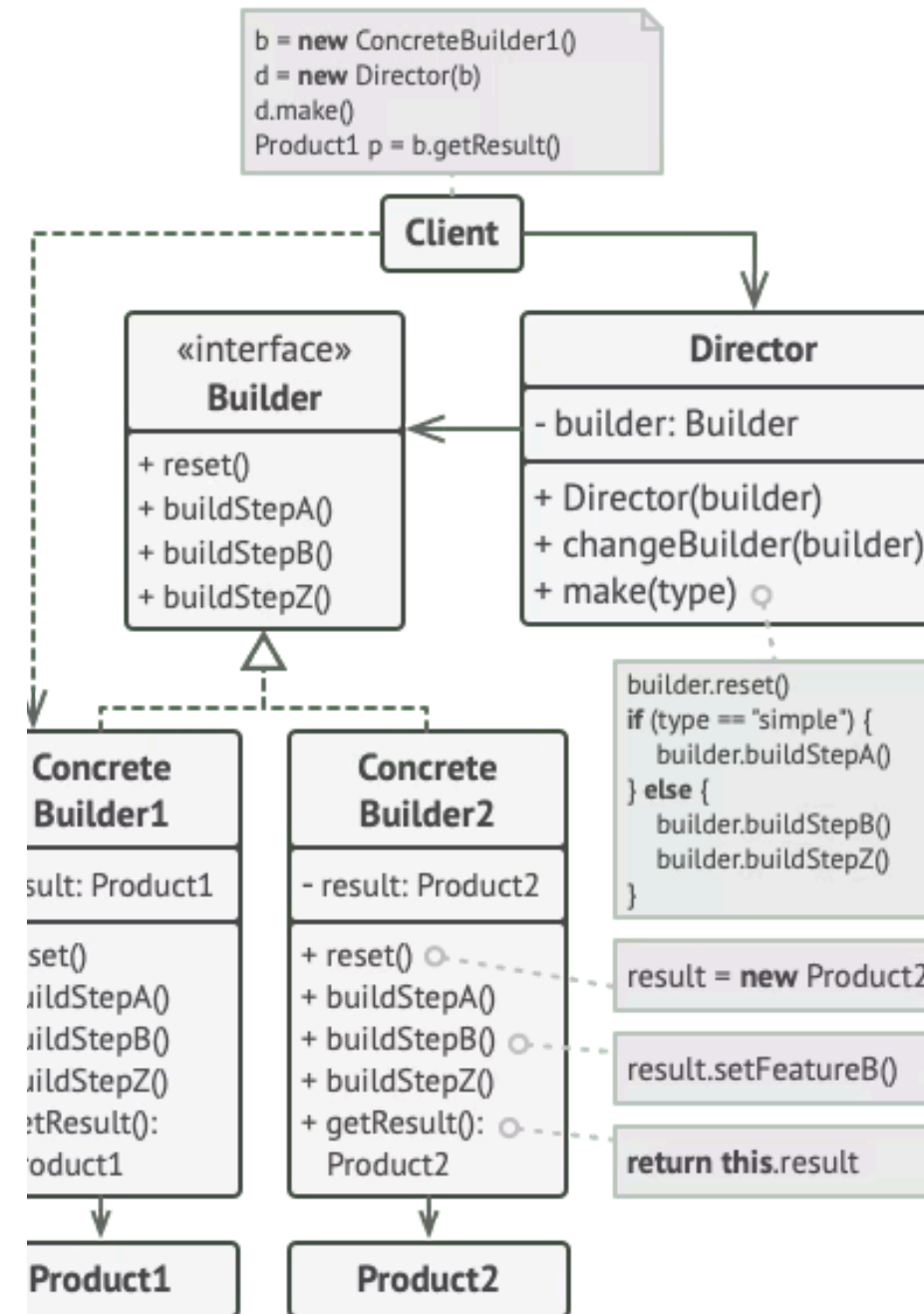
Encapsulamiento

La directora esconde por completo los detalles de la construcción al código cliente.



La clase directora sabe qué pasos de construcción ejecutar para lograr un producto que funcione.

Estructura del Patrón Builder



Componentes de la Estructura

1 Interfaz Constructora

Declara pasos de construcción de producto que todos los tipos de objetos constructores tienen en común.

2 Constructores Concretos

Ofrecen distintas implementaciones de los pasos de construcción. Pueden crear productos que no siguen la interfaz común.

3 Productos

Son los objetos resultantes. Los productos contruidos por distintos constructores no tienen que pertenecer a la misma jerarquía de clases o interfaz.

4 Clase Directora

Define el orden en el que se invocarán los pasos de construcción, permitiendo crear y reutilizar configuraciones específicas.

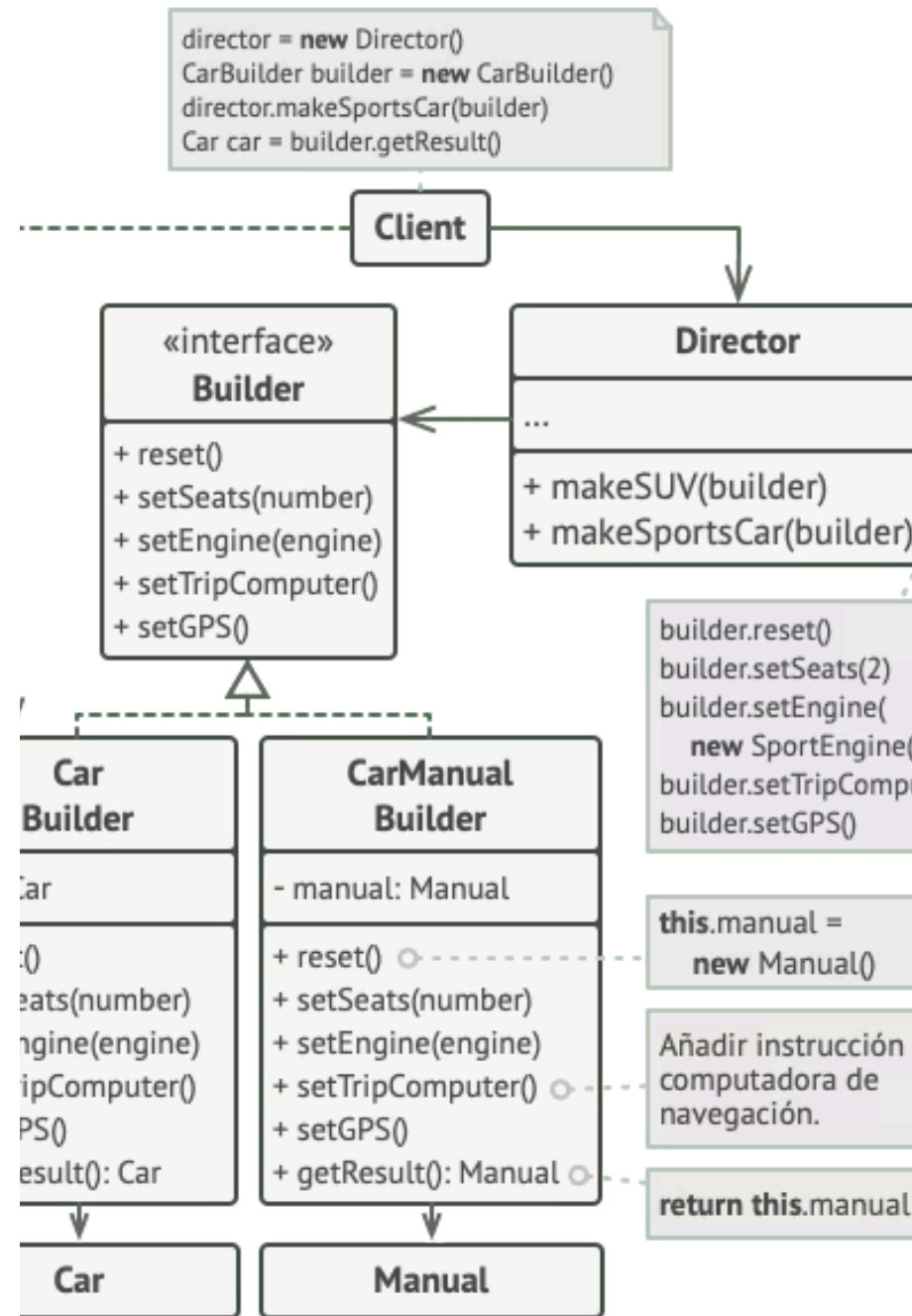
5 Cliente

Asocia uno de los objetos constructores con la clase directora, normalmente una sola vez mediante los parámetros del constructor de la directora.

Pseudocódigo: Ejemplo con Automóviles

Este ejemplo ilustra cómo se puede reutilizar el mismo código de construcción de objetos a la hora de construir distintos tipos de productos, como **automóviles**, y crear los correspondientes **manuales** para esos automóviles.

Ejemplo de una construcción paso a paso de automóviles y de los manuales de usuario para esos modelos de automóvil.



Contexto del Ejemplo

Un automóvil es un objeto complejo que puede construirse de mil maneras diferentes. En lugar de saturar la clase `Automóvil` con un constructor enorme, extrajimos el código de ensamblaje y lo pusimos en una **clase constructora independiente**.

Constructor directo

Si el código cliente necesita ensamblar un modelo con ajustes especiales, puede trabajar directamente con el objeto constructor.

Delegación al Director

El cliente puede delegar el ensamblaje a la clase directora, que sabe cómo construir los modelos más populares.

Manuales reutilizados

Otra clase constructora especializada en manuales implementa los mismos métodos, pero describe las piezas en lugar de fabricarlas.

Pseudocódigo: Productos (Car y Manual)

Los dos productos están relacionados, aunque no tienen una interfaz común.

```
// El uso del patrón Builder sólo tiene sentido cuando tus  
// productos son bastante complejos y requieren una  
// configuración extensiva. Los dos siguientes productos están  
// relacionados, aunque no tienen una interfaz común.
```

```
class Car is  
    // Un coche puede tener un GPS, una computadora de  
    // navegación y cierto número de asientos. Los distintos  
    // modelos de coches (deportivo, SUV, descapotable) pueden  
    // tener distintas características instaladas o habilitadas.
```

```
class Manual is  
    // Cada coche debe contar con un manual de usuario que se  
    // corresponda con la configuración del coche y explique  
    // todas sus características.
```


Pseudocódigo: Interfaz Builder

La interfaz constructora especifica métodos para crear las distintas partes de los objetos del producto.

```
// La interfaz constructora especifica métodos para crear las
// distintas partes de los objetos del producto.
interface Builder is
    method reset()
    method setSeats(...)
    method setEngine(...)
    method setTripComputer(...)
    method setGPS(...)
```

Pseudocódigo: CarBuilder

Las clases constructoras concretas siguen la interfaz constructora y proporcionan implementaciones específicas de los pasos de construcción.

```
class CarBuilder implements Builder is
  private field car:Car

  constructor CarBuilder() is
    this.reset()

  method reset() is
    this.car = new Car()

  method setSeats(...) is
    // Establece la cantidad de asientos del coche.

  method setEngine(...) is
    // Instala un motor específico.

  method setTripComputer(...) is
    // Instala una computadora de navegación.

  method setGPS(...) is
    // Instala un GPS.

  method getProduct():Car is
    product = this.car
    this.reset()
    return product
```

- ❏ Tras devolver el resultado final al cliente, una instancia constructora debe estar lista para empezar a generar otro producto. Por eso es práctica común invocar `reset()` al final de `getProduct()`.

Pseudocódigo: CarManualBuilder

Al contrario que otros patrones creacionales, Builder permite construir productos que no siguen una interfaz común.

```
class CarManualBuilder implements Builder is
  private field manual:Manual

  constructor CarManualBuilder() is
    this.reset()

  method reset() is
    this.manual = new Manual()

  method setSeats(...) is
    // Documenta las características del asiento del coche.

  method setEngine(...) is
    // Añade instrucciones del motor.

  method setTripComputer(...) is
    // Añade instrucciones de la computadora de navegación.

  method setGPS(...) is
    // Añade instrucciones del GPS.

  method getProduct():Manual is
    // Devuelve el manual y rearma el constructor.
```

Pseudocódigo: Director y Cliente

```
class Director is
  method constructSportsCar(builder: Builder) is
    builder.reset()
    builder.setSeats(2)
    builder.setEngine(new SportEngine())
    builder.setTripComputer(true)
    builder.setGPS(true)
```

```
  method constructSUV(builder: Builder) is
    // ...
```

```
class Application is
  method makeCar() is
    director = new Director()
```

```
    CarBuilder builder = new CarBuilder()
    director.constructSportsCar(builder)
    Car car = builder.getProduct()
```

```
    CarManualBuilder builder = new CarManualBuilder()
    director.constructSportsCar(builder)
    // El producto final a menudo se extrae de un objeto
    // constructor, ya que el director no conoce y no
    // depende de constructores y productos concretos.
    Manual manual = builder.getProduct()
```

Aplicabilidad del Patrón Builder

1

Constructor Telescópico

Utiliza Builder para evitar constructores con diez parámetros opcionales. Builder permite construir objetos paso a paso, utilizando tan solo aquellos pasos que realmente necesitamos.

2

Distintas Representaciones

Úsalo cuando quieras que el código sea capaz de crear distintas representaciones de ciertos productos (ej. casas de piedra y madera). La interfaz base define todos los pasos posibles; los constructores concretos los implementan de forma diferente.

3

Objetos Complejos (Composite)

Úsalo para construir árboles con el patrón Composite u otros objetos complejos. Puedes aplazar la ejecución de ciertos pasos e incluso invocar pasos de forma recursiva. Un constructor no expone el producto incompleto.

Ejemplo: Constructor Telescópico

Digamos que tenemos un constructor con diez parámetros opcionales. Invocar a semejante bestia es poco práctico, por lo que sobrecargamos el constructor y creamos varias versiones más cortas:

```
class Pizza {  
    Pizza(int size) { ... }  
    Pizza(int size, boolean cheese) { ... }  
    Pizza(int size, boolean cheese, boolean pepperoni) { ... }  
    // ...  
}
```

Crear un monstruo semejante sólo es posible en lenguajes que soportan la sobrecarga de métodos, como C# o Java. El patrón Builder elimina la necesidad de apiñar decenas de parámetros dentro de los constructores.

Cómo Implementar el Patrón Builder

01

Definir pasos comunes

Asegúrate de poder definir claramente los pasos comunes de construcción para todas las representaciones disponibles del producto.

03

Crear constructores concretos

Crea una clase constructora concreta para cada representación e implementa sus pasos. No olvides un método para extraer el resultado.

05

Código cliente

El cliente crea el constructor y el director, pasa el constructor al director, inicia la construcción y obtiene el resultado del constructor.

02

Declarar interfaz base

Declara estos pasos en la interfaz constructora base.

04

Crear clase directora

Puede encapsular varias formas de construir un producto utilizando el mismo objeto constructor.

06

Obtener el resultado

El resultado solo se puede obtener directamente del director si todos los productos siguen la misma interfaz. De lo contrario, el cliente debe extraerlo del constructor.

Pros y Contras

✓ Ventajas

- Puedes construir objetos paso a paso, aplazar pasos o ejecutar pasos de forma recursiva
- Puedes reutilizar el mismo código de construcción al construir varias representaciones de productos
- **Principio de responsabilidad única:** puedes aislar un código de construcción complejo de la lógica de negocio del producto

✗ Desventajas

- La complejidad general del código aumenta, ya que el patrón exige la creación de varias clases nuevas

Relaciones con Otros Patrones

Factory Method → Builder

Muchos diseños empiezan con Factory Method (menos complicado) y evolucionan hacia Abstract Factory, Prototype o Builder (más flexibles, pero más complicados).

Builder vs Abstract Factory

Builder construye objetos complejos paso a paso. Abstract Factory crea familias de objetos relacionados y devuelve el producto inmediatamente. Builder permite pasos adicionales antes de extraer el producto.

Builder + Composite

Puedes utilizar Builder al crear árboles Composite complejos, programando sus pasos de construcción para que funcionen de forma recursiva.

Builder + Bridge

La clase directora juega el papel de la abstracción, mientras que diferentes constructoras actúan como implementaciones.

Builder como Singleton

Los patrones Abstract Factory, Builder y Prototype pueden todos ellos implementarse como Singletons.